SCHOOL OF COMPUTER SCIENCE · UPES DEHRADUN

SpeakQL

Complete Logic · after Revision 5

One document, read end to end, and you understand how every part of the system decides what it decides. The specifications say what to build; this says what happens and why, in the order it happens, with worked examples. Read it before the review, and again the night before the viva. Covers the system as it stands after Backend Plan Revision 5.

Study document for the project team.

Minor Project · MCA (AI & ML)
Garv Bhargava · Dhruv Chhatrawal · Kritant Poudnel
Mentor: Mr. Pankaj Dadure

1. The system in one paragraph

A person types or speaks a question in plain English. The system works out who they are and which databases they may touch, rewrites their question into a self-contained one using the conversation so far, finds the handful of tables the question is actually about, generates SQL with a model trained for this job, refuses to run anything that is not a single read-only SELECT, decides whether its own answer is good enough or whether to ask a bigger model, runs the query under a database account that can only read, picks a chart from the shape of the result, explains the answer in words naming the columns it came from, and writes down everything it just did so the accuracy can be measured later. If the answer is short because rows are missing, it says so and lets the right person fill the gap where they are standing — and their own answers use their correction from that moment, while everybody else keeps seeing verified data. If the question was ambiguous it asks back once instead of guessing. If the data cannot answer it, it says so.

Everything else in this document is the detail of that sentence.

2. Vocabulary — define these without hesitating

Panels test vocabulary. Stumbling on "what is an embedding" undoes a good demo.

TERM	SAY THIS
Embedding	A list of numbers standing for a piece of text, arranged so similar meanings sit close together
Bi-encoder	A model that embeds two things separately so they can be compared by distance — fast, because columns are embedded once in advance
FAISS	A library that finds the nearest vectors among millions in milliseconds
Schema linking / retrieval	Working out which tables and columns a question is about, before generating any SQL
Spider	A public benchmark: about 7,000 questions with gold SQL over 140 databases, split so test databases are unseen
CodeT5	An encoder-decoder model pre-trained on source code; we fine-tune the 60M version to write SQL
Fine-tuning	Continuing training of a pre-trained model on your own task data
Beam search	Decoding that keeps several candidate outputs alive and returns the best-scoring one
Confidence	The beam score divided by output length, turned into a 0–1 number. Length-normalised, because longer outputs always score lower otherwise. **It is not a probability that the SQL is right** — see §5
AST	Abstract syntax tree — the parsed structure of a query, which we inspect instead of pattern-matching the text
sqlglot	The library that parses SQL into that tree
Product role	Who you are in your company: owner or member. There is nothing above it
Database role	What you may do with one database: analyst or viewer. Stored on the grant, not the user
Access grant	The row that makes one database appear for one person, at one database role
Persona	Answer depth: Workbench gets chart, explanation and SQL; Readout gets the number and its rows. Never changes what may run
Personal workspace NEW	The organisation a free-mail signup gets: exactly one person, and `domain` NULL so that domain resolution can never match it (§3.2)

TERM	SAY THIS
Overlay	A small schema holding one member's corrections that have not been merged yet. Not a copy of the warehouse
Overlay view NEW	The `COALESCE` view that lays a member's pending rows over the real table so their own answers include them. Read-only, and only ever read by its own author (§7.4)
Merge request	A member's proposed correction, waiting for the owner to approve or reject
Stale merge NEW	A merge request whose row was changed by somebody else after it was raised. It is refused rather than applied (§7.6)
Object-level authorisation NEW	Checking that an identifier in a request — a message, a thread, a merge — belongs to the person asking, separately from checking their database grant (§3)
Clarification	The pipeline asking back with 2–4 options instead of guessing
Execution accuracy	Run our SQL and the gold SQL; do the result sets match? Our primary metric
Valid-query rate	Does the query parse and execute without error?
Recall@k	Of the tables the gold query needed, how many were in our top k retrieved?
Ablation	Removing one component and re-measuring, to prove it contributes

3. Permission logic — how a request is authorised

Every request answers **five** questions in order. Failing any one stops the request. Revision 4 had four; the fourth is new, and it is the one a panel is most likely to probe, because it is the check most projects forget.

```
1. Is the session token valid? no -> 401
2. Is the account active (not pending)? no -> 403
3. Is there a grant for THIS database? no -> 403
4. Is every id in this request theirs to name? no -> 403
5. Does the grant's database role allow the action? no -> 403
```

Step 5 is where the two role axes meet:

BEING ASKED	NEEDS
Read a table flagged `is_public`	Any grant
Read any other table	analyst grant
See the generated SQL	analyst grant, and persona workbench
Correct a value	analyst grant
Have that correction land in the real table immediately	owner
Approve someone else's correction	owner
Invite, approve a signup, grant access	owner

> **The invariant to memorise.**
> The organisation and connection identity come from the session, never from the request body — and are checked again in the executor immediately before the query runs. Two checks, because one check is one point of failure.

Why two role axes at all? Because "who you are in the company" and "what you may do with one particular database" are different facts. The same person can be an analyst on Sales and a viewer on Finance. One axis could not express that.

Why only two product roles? Because a third one that could see across companies would be the only thing in the system able to break tenant isolation, and it existed to serve a support workflow we are not building. Removing it removes the risk entirely rather than guarding it.

3.1 Step 4, and why it is separate

New in Revision 5. Steps 3 and 5 are about a *database*. Step 4 is about an *object*. They are different questions and they fail for different reasons, so conflating them leaves a hole.

Suppose a member holds a perfectly valid analyst grant on `northwind_sales`. Step 3 passes and step 5 passes. Now they call `GET /api/ask/8f2c.../export` with a message id belonging to somebody in another company. Without step 4, every check they face is about a database they legitimately have — and none of them is about the message they are actually asking for.

IDENTIFIER	RESOLVED AGAINST	REFUSED WHEN
`message_id`	`messages` → `threads.person_id`	The thread is not theirs
`thread_id`	`threads.person_id`, `organisation_id`	Different person, or different organisation
`merge_id`	`merge_requests.organisation_id`	Not the owner of the organisation that raised it
`connection_id`	`access_grants`	No grant, or revoked
`dataset_id`	`uploaded_datasets.organisation_id`	Different organisation

> **The detail worth volunteering.**
>
> An object that is not yours and an object that does not exist return the *same* 403. If a foreign id returned 403 and a nonexistent one returned 404, anybody could map out which ids exist by iterating them. Saying this unprompted is the kind of answer that tells a panel you thought about the attack and not just the rule.

3.2 How the role is decided, once

This is the part reviewers ask about, because it is the only place a person chooses their own permissions.

SIGNING UP WITH	WHAT HAPPENS	ASKED ANYTHING?
A domain nobody has registered	They are told nobody from that domain is here yet, and asked whether they are setting SpeakQL up or joining a team. "Setting up" creates the organisation and makes them its owner.	Yes, once
A domain already registered	They join that organisation as a member, pending the owner's approval.	No
An invitation token	The token fixes the organisation, the role and the database grants. The account is created active. The token is single-use and expires after a week.	No
A free-mail address	Accepted, and given a **personal workspace** — an organisation of one, with no domain at all. No company question, because there is no company.	No

Answering "I was invited" on a company domain nobody has registered is a dead end by construction: there is no organisation to join, and an invitation cannot conjure one, because an invitation is issued by an organisation. So exactly one owner exists per company domain, created at the moment that domain is first seen, by somebody who said so.

Why gmail is allowed, and why that is not a hole

Have this one ready, because it sounds like a contradiction. An earlier revision refused free-mail outright on the grounds that a company is identified by its domain. That reasoning was right about the **lookup** and wrong about the **address**.

Think about what allowing it naively would do. `resolve_domain('gmail.com')` would return one organisation, and **every Gmail user in the world would be dropped into it** — one tenant, everybody reading everybody else's warehouse. That is the worst failure this system could have, and it is why the refusal existed in the first place.

So the rule is not "allow gmail". The rule is: **a free-mail address never resolves to an organisation by domain.** A free-mail signup gets a personal workspace whose `domain` column is NULL, and the resolver asks `WHERE domain = ? AND kind = 'company'`. A null never equals anything, so that workspace is unreachable by domain — forever, for everybody, including its own owner.

QUESTION A PANEL WILL ASK	ANSWER
Two people sign up with Gmail. What happens?	Two separate organisations. Neither appears to the other, on any screen or in any query. There is a test that asserts exactly this.
So how does anyone else get into a personal workspace?	An invitation issued from inside it. Explicit, single-use, and it names the organisation — so no lookup is involved at any point.
Can a company invite somebody on Gmail?	Yes, and this is the case the old refusal could not serve: a contractor. The token fixes the organisation, so again no domain lookup happens.
Does a personal workspace get weaker rules?	No. Same two product roles, same database roles on grants, same validator, same privileged path register. Nothing downstream of signup knows or cares which kind it is.
What if an installation wants the old behaviour?	`FREE_EMAIL_MODE=blocked` restores it. The default is `personal_workspace`.

> **The one-sentence version.**
> The danger was never the address — it was letting a free-mail domain resolve to an organisation. Keeping that lookup company-only is one line, it removes the danger completely, and it costs the product nothing, because a workspace of one does not need a domain.

4. Question logic — from typed words to a self-contained question

Two things happen before anything is generated. Neither can widen what the grant already allows — they only narrow or pause.

4.1 Context resolution

"Now break that down by region" means nothing alone. The resolver reads the last few turns — the previous question, the SQL that answered it, the filters it used — and rewrites the follow-up into a standalone question carrying the same metric, tables and date range, plus the new grouping.

Both versions are stored: what the person typed, and what the system understood. When an answer is wrong, this is how you find out whether the model misread the question or misread the schema.

4.2 Retrieval

The question is embedded and matched against a vector index of every column in that organisation. Top matches expand to their parent tables, plus one hop along foreign keys, then get filtered by the person's database role.

The filtering order matters: **retrieve first, restrict second.** Restricting first would mean re-embedding per role.

One thing to know about the index: an uploaded file is embedded and indexed as the last step of ingestion, synchronously, before the upload reports success. Until Revision 5 it was not, which meant a table you had just loaded could not be found by a question — the demonstration script walks straight into that four steps after the upload, so it is worth knowing why the fix exists.

4.3 Clarification

If two candidate metrics score similarly, or generation confidence lands in the uncertain band, the pipeline stops and offers 2–4 readings. The person picks; generation re-runs with that choice folded in.

One round only. Still ambiguous after that, the system answers with its best guess and states the assumption in the explanation. One clarification is good service; three is a broken interface.

There used to be a third step here: a business glossary that mapped a company phrase to a SQL expression. It is gone. It was a second place for the meaning of a word to live — needing its own authoring screen, its own validation and its own failure mode — and the explanation already names the columns a number came from, which lets the reader check the definition at the only moment they care about it: when they are looking at the answer.

5. Generation and routing logic

The generator receives the question plus **only the retrieved schema subset** — not the whole 18-table schema. Two reasons: shorter input decodes faster and more accurately, and restricting the input is exactly what makes retrieval matter. Feeding the full schema is kept as an ablation condition to prove that.

Beam search returns the best candidate and a score. The score is divided by the number of tokens, because raw sequence probability always falls as output grows — without normalising, the router would systematically prefer short queries, which on this task means queries missing a join.

```
generate -> candidate SQL + confidence
validate -> accept / reject
if valid AND confidence >= threshold -> run locally (no API call)
else -> regenerate with Gemma, validate again
```

The fallback is Gemma — `gemma3:4b`, Google's open-weight model, running locally in the `llm` container under Ollama. It has exactly two jobs: generate SQL when CodeT5 is not confident enough, and write the explanation in step 11, which CodeT5 writes badly. Know the name and the size; "a larger model" is not an answer to "which model?".

Until CodeT5 is fine-tuned, Gemma is the generator for every question. That is the honest state of the second presentation, and it is the second row of the five-configuration study in the synopsis — so say it before you are asked. The router, the threshold and the escalation arm are all built; there is simply nothing below the threshold to escalate *from* yet.

The threshold is not guessed. It is swept across the development set, plotting share of questions served locally against execution accuracy. The curve has a knee — past it, serving more locally costs accuracy disproportionately. That point is chosen, and the plot goes in the results chapter.

> **The confidence question, and the honest answer.**
>
> A panel may ask whether confidence means the query is probably correct. It does not. It is the model's confidence in its own decoding — how sure it was about the tokens it chose. That correlates with correctness, which is why it works as a routing signal, but we have not measured calibration and so we do not claim it. Volunteering that distinction is worth more than a confident wrong answer about it.

Why keep a fallback at all? Because a 60M-parameter model genuinely cannot match a frontier model on hard, unseen queries, and pretending otherwise builds a system that fails silently. Measuring how often we need help is a result, not an

embarrassment.

Why not use the fallback for everything? Cost, latency, and the schema leaving the network on every request. And a project that only calls an API is integration work, not machine-learning work.

6. Safety logic — four layers and what each catches

LAYER	CATCHES	WHERE
1. Intent gatekeeper	A question asking for credentials or destruction, before any SQL exists	Before generation
2. AST validator	A generated statement that is not a single read-only SELECT over known, permitted tables	After generation, before execution
3. Database role	Anything that slipped past layer 2 — the read account simply has no write privilege	At the database
4. Edit validator	A correction against a non-editable table, or one not scoped to a single row	On the write path only

Validation happens on the parsed tree, not the query text, so formatting, casing and comment tricks cannot evade it.

The read rules:

- parses under sqlglot
- exactly one statement
- statement type is SELECT; no DDL or DML node anywhere in the tree
- every referenced table and column exists in this organisation's registry
- every referenced table is permitted for this database role
- no comment construct hiding a second statement

And the write rules, which are stricter and shorter:

- the table is on the editable list — which means it has a primary key and the owner has not marked it read-only
- the statement is scoped by that primary key
- exactly one row is affected, or the transaction rolls back
- the person holds an analyst grant on that database

Worked attacks:

INPUT	CAUGHT BY	WHAT HAPPENS
"Delete all orders from last year"	Layer 2, statement type	Rejected, rule named, nothing executed
"Show everything; DROP TABLE orders;"	Layer 2, statement count	Rejected
"What is the database password?"	Layer 1, intent	Rejected before generation
A reference to another company's table	Layer 2, registry membership	Rejected, 403 logged
Another company's `message_id` in an export URL	Step 4 of §3, object access	Rejected — 403, identical to a nonexistent id, and an audit row
A correction with no primary-key scope	Layer 4, scoping rule	Rejected
A correction on a table with no primary key	Layer 4, editable list	Rejected — that table is read-only for everyone
A correction by a viewer	Layer 4, grant check	403

INPUT	CAUGHT BY	WHAT HAPPENS
Hypothetically, a write that passed everything on a read connection	Layer 3	Fails at the database — `speakql_ro` has no write privilege

> **The line worth saying out loud.**
>
> The read path holds no write privilege of any kind, and every write in the system goes through one path that is validated, scoped to a single primary key, restricted to tables the owner marked editable, and written to the edit log.

Be ready for the follow-up. A panel may point out that an earlier version of this project claimed the application never writes to production at all. That was true when nobody could correct anything, and it stopped being true the moment editing was added. Saying it anyway would be a claim the panel could take apart in one question. The sentence above is the strongest one that is actually true, and the six assertions in §17 of the Backend Plan are what make it checkable rather than asserted.

6.1 A fifth rule, which is not a layer

New in Revision 5. The four layers protect the database. Nothing in them protects the *explainer*.

`explainer.py` receives rows read out of a customer's warehouse and writes a sentence about them. A cell containing "ignore your instructions and ..." is untrusted input arriving at a language model. It is not a SQL problem, so no validator catches it. Three rules handle it:

- Result rows go into a **separate, delimited data section** of the prompt, never concatenated into the instruction section.
- The explainer's output is **text only** — it cannot cause a query, a write, an email or a tool call. This is the structural half of the defence and it is the half worth saying first, because it bounds the worst case to "a misleading sentence" rather than "an action".
- The column names in an explanation come from `schema_registry`, not from the model. So the checkable part of an explanation cannot be forged by data.

Why this is not called layer 5: the four layers all answer "may this run?". This one answers "what may this text do?", which is a different question with a different failure. Numbering it alongside them would suggest the validator catches it, and it does not.

7. Data logic — where data lives and who may change it

PLACE	WRITTEN BY	READ BY	PURPOSE
The warehouse	The owner, through one validated single-row path	`speakql_ro`	The source of truth
A member overlay	That one member, through the same validator	That member only, through their overlay view, until merged	Corrections proposed but not yet accepted
Uploads database	File ingestion (`speakql_upload_ddl`)	`speakql_ro`	Tables created from CSV or Excel files

7.1 Where a gap is noticed

Incompleteness is a property of the data, so it shows up in the answer. A row with no value for a summed column is left out of the total rather than counted as zero, the answer says how many rows that was, and the table underneath marks each missing cell. For anyone holding an analyst grant, that cell is also the way to fill it.

> **The whole design idea, in one sentence.**
> The place you notice a gap is the place you fix it. There is no mode to enter and no toggle to remember.

7.2 An owner's correction

```
validate -> editable table? scoped by primary key? analyst grant?
write -> speakql_write, inside a transaction
check -> exactly one row affected, or roll back
log -> edit_log: before, after, who, note
respond -> the corrected row; the open answer recalculates
```

No approval step. An owner approving their own change is theatre, and a step that exists only to be clicked through trains people to click through steps.

7.3 A member's correction

```
validate -> the same three checks
write -> speakql_edits_rw, into that member's overlay
raise -> merge_requests: row key, before, after, note
notify -> the owner, digested; then a reminder each morning while open
meanwhile -> that member's own answers include their value and say so;
             everybody else sees the real data
```

Why a pending value appears in their own answers and nobody else's. They are the person who knows the real figure, and making them wait on an approval to use knowledge they already have is what drives people back to spreadsheets. Everyone else sees verified data, which is what protects the numbers other people quote. The answer is labelled either way, so nobody is misled about which they are looking at.

7.4 How that actually works — the overlay view

New in Revision 5, and the single most likely "how?" question in the viva. The previous section promises that a member's own answers include their pending value. Here is the mechanism, because "it just does" is not an answer.

The obvious approach is wrong, and knowing why is the interesting part. You cannot run the query against the real table and then patch the result, because **the answer is an aggregate**. If the query was `SUM(units)` and the member filled in a missing `units`, patching means recomputing the SUM in Python — and then recomputing every other aggregate the generator can emit, for every query shape, correctly, forever. Get one wrong and the number is silently off.

So the correction is applied **before** the arithmetic, by the database, using a view:

```
CREATE VIEW member_edits.v_<person>_<table> AS
SELECT p.<pk>,
       COALESCE(e.<col>, p.<col>) AS <col>, -- one per editable column
       ...
FROM public.<table> p
LEFT JOIN member_edits.e_<person>_<table> e USING (<pk>);
```

`COALESCE` takes the member's value where they have one and the real value everywhere else. The executor points that member's query at this view instead of `public.<table>`, and the SUM runs over corrected rows. Same arithmetic, different rows.

QUESTION A PANEL WILL ASK	ANSWER
Does this give the read path write access?	No. `speakql_ro` gains SELECT on the *views*, and nothing else — not on the overlay tables underneath them, and never anything but SELECT. The assertion that protects the warehouse runs the other way and is untouched.
Who sees the view?	Only its author. It is created per member, and the executor resolves it only on that member's own request. An owner never resolves an overlay — owners read `public`, always.
Does the model see pending data?	No. The generator always produces SQL against `public`. The rewrite happens in the executor, after generation, so nothing about a pending value reaches the prompt.
When is it cleaned up?	The view is created on the member's first correction to that table and dropped when their last pending row for it is merged or rejected. A member with nothing pending has no views at all.
Does this pollute the accuracy figures?	No, and this is the point of `used_pending`. The rewrite sets that flag, and every accuracy figure filters it out (§9).
What stops it growing forever?	`OVERLAY_MAX_TABLES_PER_MEMBER`, default 25. Somebody with pending corrections across twenty-five tables has a problem their owner needs to see, and silently growing the schema would hide it.

7.5 The merge

`POST /api/merges/{id}/resolve` takes one of two decisions. **Approve** applies the value to the row the request names, under `speakql_write`, writes an `edit_log` row and tells the member. **Reject** drops it from the overlay and tells the member why. The request carries the row key it was raised against and cannot be redirected at a different row.

The failure this feature actually has is not a wrong merge. It is a forgotten one — a correction sitting unreviewed for a week while the person who made it assumes it landed. That is what the daily reminder exists for, and it is worth saying so when asked why there is a scheduler in the stack.

7.6 The staleness check, and the bug it fixes

New in Revision 5. This is worth rehearsing, because it is the one place the project found a real correctness bug in its own design and fixed it — and panels reward that more than a feature.

The bug. A member proposes 42 for a row. The owner, independently, corrects that same row to 50 — which lands immediately, because owners write directly. The owner then works through their merge queue and approves the member's request. Under Revision 4, 42 overwrites 50. A verified value is destroyed by an older proposal, and **both writes look completely legitimate in the edit log**, so nothing surfaces it.

The fix. Every merge request already records the `before` value it was raised against. At resolution:

```
read -> the live row, under speakql_write, INSIDE the same transaction
compare -> live value vs the before value on the request
equal -> apply, log, tell the member (the Revision 4 path)
differs -> write nothing; state = stale; 409
           show the owner all three values: was / proposed / is now
```

The read has to be inside the same transaction as the write. Checking first and writing afterwards leaves a window in which the row can move between the two, which is the same bug wearing a smaller hat.

The same rule covers two members proposing different values for one row. The first approval succeeds; the second finds the row changed and goes stale, so the second member is told what happened rather than the first member's approved value vanishing without either of them knowing.

> **Stale is not an error state.**
> It is the system noticing that two people disagreed about one number — exactly the thing worth putting in front of an owner, and exactly the thing a silent last-write-wins would have buried. If asked "what would you have missed without it?", the answer is: nothing visible. That is what made it dangerous.

7.7 What the staging mirror was, and why it is gone

Revision 3 kept a writable copy of the whole warehouse. Edits went there, questions carried a `source` parameter to choose which version to read, a nightly job re-synced the copy, shadow hashes detected rows that had changed on both sides, and conflicts had a three-state machine and two endpoints.

It was removed for one reason: it asked every person to hold two versions of the truth in their head and choose between them on every question. Almost nobody would, and because the default was production, most people would never have seen their own team's corrections at all. Everything else — the resync, the hashes, the drift fingerprints, the conflict machine — existed only to keep that copy honest. With no copy, none of it is needed. It is the single largest reduction in moving parts in the project, and it is a good answer to "what did you cut, and why?"

Worth noticing: the overlay view in §7.4 is what a staging mirror wanted to be. It is per person, holds only pending rows rather than a copy of everything, needs no sync because it reads through to the real table, and requires no choice from the person using it.

8. Answer logic

Chart choice comes from the shape of the result, not from the question:

RESULT SHAPE	CHART
One number	Metric card with a delta
One category (12 or fewer) and one number	Bar
A date column and a number	Line
Parts of a whole	Donut
Two numbers	Scatter
Signed contributions to a change	Diverging bars
Anything else	Table only

Explanation names the columns the number came from and states the arithmetic in one clause, capped at about sixty words. Naming the source column is what makes the answer checkable rather than merely readable — and with the glossary gone, it is now the only place a reader can check what a word was taken to mean.

8.1 What each persona gets

	READOUT	WORKBENCH
On screen	The number, the explanation, and the complete table of rows behind it	The number, a chart, the explanation, the SQL, and the complete table of rows
The data	Every row. Persona never truncates data	Every row
SQL	Stripped from the response body server-side, not hidden by the frontend	Included
Export	CSV of every row	CSV of every row, and a report carrying the charts and the analysis

Persona decides depth, never permission. A viewer is Readout and cannot change it, because a viewer may not see SQL at all.

> **If a panel asks why the names changed.**
>
> Because `analyst` was already a database role. Somebody could hold a **viewer** grant and read in **Analyst** mode, and the sentence describing that was gibberish. The modes are now **Workbench** and **Readout** — the instrument or the measurement — and `analyst` means exactly one thing again. Readout removes the chart and the SQL; it never removes a row.

8.2 Five ways the system does not give a plain answer

These matter more than the happy path. A refusal that looks like an answer is a lie in interface form.

MODE	TRIGGER	SHOWN
Blocked	Validator rejected the query	Danger card naming the rule that fired
Clarification	Several readings are defensible	2–4 options, no chart, no number
Refusal	The data cannot support the question	Reason, plus what data would be needed
Low confidence	Answered, but below threshold	Amber strip stating the assumption made
Incomplete	Rows are missing values the answer needed	The number, marked short, with how many rows were excluded and a way to fill them

The fifth changes behaviour rather than just wording: it turns "this number looks wrong" into a thing the reader can act on without leaving the answer.

A sixth state exists but is not a response mode: the answer is still **arriving**. The pipeline streams one stage event per step, so the interface names the step it is on rather than showing a spinner. It is worth knowing this exists, because "what does the user see for the two seconds the model is running?" is a fair question and "nothing" is a poor answer.

9. Measurement logic

Three tables, each with exactly one writer: `query_log`, `edit_log`, `audit_log`, plus `feedback`. `query_log` gets a row on every path — answers, rejections, refusals, rate limits, clarifications never resolved, and crashes. A request that does not log is a request that never happened as far as the results chapter is concerned.

```
every accuracy figure = computed over mode = 'read' AND used_pending = false
```

An answer that included the asker's own unmerged correction is answering over data they typed in. Counting it would overstate the model. This is the same protection Revision 3 got from `source = 'production'`, expressed against the new design — and `used_pending` is set by the overlay rewrite itself (§7.4), so it is exact rather than inferred.

Two further figures are reported separately and honestly: **clarification rate**, and **the share of questions served locally**. Clarification rate matters more than it did in Revision 3 — with no glossary, it is the main evidence that retrieval is improving.

10. Six worked traces

Memorise these. Almost any viva question is answerable by pointing at a step of one.

Trace A — a confident question

"Which region had the highest sales last quarter?"

- Session gives person, organisation, database role analyst. No foreign ids in the request.
- No pronouns, so the context resolver passes it through.
- Retrieval returns `orders`, `customers`, `regions` — the model learned "sales" means `orders.amount`.
- Candidate metrics are not ambiguous, so no clarification.
- CodeT5 generates a three-table join with SUM and GROUP BY; confidence 0.94.
- Validator: one statement, SELECT, all tables known and permitted.
- 0.94 is above the threshold, so local. No API call.
- Executes under `speakql_ro`, 10-second timeout, 5,000-row cap. This person has nothing pending, so no overlay is resolved.
- One category and one number, so a bar chart.
- Explanation: "West led Q4 with 482,140, about 36% ahead of North, summed from `orders.amount` across 1,284 rows."
- `query_log`: route=local, mode=read, used_pending=false, 412 ms.

Trace B — an ambiguous question

"Which was our best month?" Steps 1–3 as above. At retrieval, three similarly-scored readings surface. The pipeline stops before generating anything and offers: highest revenue, fastest growth, best margin. Three readings give three different months — June, April and March respectively. The person picks growth; generation re-runs with that choice; the answer is April. A `clarifications` row records the exchange.

The point: guessing silently would have produced a confident, wrong answer with nothing to signal it.

Trace C — a blocked question

"Delete all orders from last year." The generator produces a DELETE. The validator rejects it at the statement-type rule; nothing is sent to the database. The person sees which rule fired. `query_log` records `valid=false` with the reason. Even if the validator had somehow passed it, `speakql_ro` holds no DELETE privilege — layer 3 would have failed it at the database.

Trace D — an owner fills a gap

"Total shipments for week 12." The answer comes back as 1,083, marked short: three rows have no unit count, and rows with no value are excluded from the total rather than counted as zero. The table underneath marks each missing cell.

- The owner clicks the missing cell for Thursday and types 203.
- The edit validator checks: `shipments` is on the editable list, the statement is scoped by `shipment_id`, exactly one row will be affected.
- The write runs under `speakql_write` inside a transaction.

- `edit_log` records empty → 203, who, and the note.
- The answer on screen recalculates to 1,286 while they are still looking at it.
- No approval step, because they are the owner.

Trace E — a member fills the same gap

The first four steps are identical, and the validator is the same code. Then it diverges:

- The row is written to that member's overlay under `speakql_edits_rw`, which holds no grant of any kind on `public`. A checksum of the real table before and after is byte-identical.
- An overlay view is created over `shipments` for this member, if they did not already have one.
- A `merge_requests` row is raised with the row key, before, after and the note.
- The owner is emailed — digested, then reminded each morning while it is open.
- The member asks again. The executor points the query at their view, `COALESCE` supplies 203, and the SUM returns 1,286. `used_pending` is set true on that log row.
- The owner asking the same question still sees 1,083 — owners never resolve an overlay.
- The owner approves. The value is applied to the real row, logged, the view is dropped because nothing is pending any more, and the member is told. Now everybody sees 1,286 from `public` directly.

> **Traces D and E are the pair to rehearse together.**
> They are the same person-facing action, the same validator and the same three checks. The only things that differ are which engine the write goes through, whether a merge request is raised, and whether the read comes back through a view. That is the whole permission model in one example.

Trace F — a merge that has gone stale

New in Revision 5, and the one to have ready when somebody asks what could go wrong. It picks up exactly where Trace E paused, before the owner approved.

- The member's request is open, proposing 203 against a `before` value of empty.
- Before working the queue, the owner happens to correct that same Thursday row directly — they have the carrier manifest in front of them and type 211. It lands immediately in `public`, with its own `edit_log` row. Nothing about this touches the member's request.
- The owner now opens the merge queue and approves the member's 203.
- Resolution reads the live row inside the transaction that would apply the change: it is 211, and the request was raised against empty. They differ.
- **Nothing is written.** The request moves to `stale` and returns 409.
- The owner sees three values side by side — was empty, proposed 203, is now 211 — and decides. Rejecting tells the member why. Applying anyway raises a fresh request against 211, so the decision is recorded against what was actually on the row.

Under Revision 4 this trace ended differently: 203 would have overwritten 211, silently, and both writes would have looked correct in the log. Being able to describe the version of your system that had the bug, and why the fix is where it is, is a stronger answer than a system that never had one.

11. Ownership and the seams

AREA	OWNER	EVERYONE MUST STILL KNOW
Models, router, pipeline	Garv	What each model does and why it is trained (§5)
Interface, product surface, docs	Dhruv	The five non-plain-answer modes (§8.2)
Data, evaluation, tests	Kritant	How synthetic pairs are verified
Auth, tenancy, validators	Garv + Dhruv	Why identity comes from the session (§3)

Three seams where bugs live:

- **Retriever to generator.** If retrieval misses a table, the generator cannot recover, and a retrieval failure looks exactly like a generation failure. When debugging a wrong query, check retrieval first.
- **Backend to frontend.** The mock and the API response must share one shape. If they drift, the interface gets rewritten. This seam now has a written contract — §12 of the Backend Plan — and a contract test the mock is checked against, which is the only reason it is no longer the most dangerous line in this document.
- **Query log to results chapter.** Incomplete logging means the evaluation chapter cannot be written.

12. Numbers to know cold

QUESTION	ANSWER
Analyst effort per ad-hoc request	2–4 hours
Queue wait	1–2 days
Tables in the warehouse	18
Spider training questions	about 7,000 across 140 databases
Synthetic in-domain pairs	about 1,500, all execution-verified
Held-out benchmark	50+ hand-written pairs
Model A	MiniLM bi-encoder, 22M parameters, about 25 minutes to train
Model B	CodeT5-small, 60M parameters, about 3–4 hours
The LLM	Gemma — `gemma3:4b` quantised, about 3 GB, local under Ollama. Not trained by us
What the LLM is never used for	Any safety decision. sqlglot decides
Expected accuracy — Spider dev	40–55%
Expected accuracy — our warehouse	70–85%
Expected accuracy — hybrid	85–92%
Local vs fallback latency	about 400 ms vs about 1.8 s
Expected local coverage	about 70%
Safety layers	4, plus the prompt rule in §6.1
Product roles / database roles	2 / 2

QUESTION	ANSWER
Organisation kinds	2 — company (has a domain) and personal (has none)
Pipeline steps	12
Authorisation questions per request	5
Merge states	4 — open, merged, rejected, stale
Privilege assertions run in CI	6

> **The accuracy trap.**
> Never say "85%" without saying on what. The correct sentence is "70–85% on our own warehouse, 40–55% on Spider dev which is unseen schemas, and 85–92% for the hybrid system we actually ship."

13. Things not to say

DO NOT SAY	SAY
"It uses AI to understand the question"	"A fine-tuned bi-encoder retrieves the schema, then a fine-tuned CodeT5 generates the SQL"
"It is 85% accurate"	Give all three settings
"It replaces analysts"	"It removes routine read requests so analysts do deeper work"
"It answers anything"	"Each question type has a defined response, including refusal"
"The AI decides if the query is safe"	"A deterministic AST parser decides. The model is never trusted for safety"
"We use an LLM" / "we use a larger model"	"Gemma — `gemma3:4b`, open-weight, running locally under Ollama. It generates below the confidence threshold and writes the explanation. Its SQL goes through the same validator as CodeT5's"
"The LLM checks the query before it runs"	"Gemma generates and explains. sqlglot decides. The safety decision is identical whether the SQL came from CodeT5, from Gemma, or from a person"
"The application never writes to the database"	"The read path holds no write privilege, and every write goes through one validated, single-row, audited path"
"Users can edit whatever they like"	"An owner corrects a row directly; a member proposes a correction the owner merges"
"The model knows when it is wrong"	"Confidence is the length-normalised decoding score. It correlates with correctness and we route on it, but we have not measured calibration and do not claim it"
"Members see their edits because we merge the results"	"Their query is pointed at a `COALESCE` view, so the database does the arithmetic over corrected rows"
"An approved merge is always applied"	"It is applied unless the row moved underneath it, in which case it goes stale and the owner decides"
"We allow gmail now"	"A free-mail address gets a personal workspace with a null domain, so it is never matched by domain resolution — two Gmail users are two separate tenants"
"That is someone else's module"	Answer at the level of §10, then say who owns the detail

14. Test yourself

Answer aloud, without notes. Section in brackets if you stumble.

- Explain SpeakQL in three sentences to a non-technical person. [§1]
- What are the two models you train, and what does each do? [§5]
- **Which LLM, what size, where does it run, and what are its two jobs?** [§5]
- **Gemma writes a DELETE. What happens, and which component stops it?** [§5, §6]
- Why not just call an LLM for everything? Three reasons. [§5]
- Walk through "which region had the highest sales" — all eleven steps. [§10A]
- Name the five ways the system does not give a plain answer, with an example of each. [§8.2]
- What are the four safety layers, and what does each catch? [§6]
- What is confidence, exactly, and what does it not tell you? [§2, §5]
- How does someone become an owner, and how many owners can one company have? [§3.2]
- **Two people sign up with Gmail addresses. Are they in the same organisation? Why not?** [§3.2]
- An owner corrects a value and a member corrects the same value. Trace both. [§10D, §10E]
- **A member fills a gap and their own total changes. How, mechanically? Why not just patch the number afterwards?** [§7.4]
- **What happens if an owner edits a row that already has a merge request against it? What would have happened before Revision 5?** [§7.6, §10F]
- How do you prove a member's correction did not touch the real table? [§7.3]
- **Someone pastes another company's message id into an export URL. What stops them, and why does it return 403 rather than 404?** [§3.1]
- You used to claim the application never writes to production. What changed, and what do you claim now? [§6]
- What did you remove between Revision 3 and Revision 4, and what did you add in Revision 5? [§7.7, §4.3]
- Difference between execution accuracy, exact match and valid-query rate? [§2]
- What stops person A reading company B's data? Name both enforcement points. [§3]
- What are your three accuracy figures, and which one ships? [§12]
- Why is a pending correction visible to its author but nobody else? [§7.3]
- **A warehouse cell contains the text "ignore your instructions". What happens, and which layer catches it?** [§6.1]
- What is the weakest part of the project, and what would fix it? [below]

> **Answer to the last one — rehearse it.**
>
> Cross-domain generalisation. On unseen Spider schemas we score 40–55%. Our strong numbers are in-domain, on the warehouse we trained for, and adapting to a new schema currently needs synthetic pair generation for that schema. That is the main direction for future work. Volunteering your own limitation, with a number attached, is the strongest signal that you ran the experiments yourself.

15. Document map

FILE	READ WHEN
This document	Now, and the night before the viva
SpeakQL Backend Plan — Revision 5	Before writing backend code. §12 is the API contract
SpeakQL Frontend Specification	Before writing frontend code, and to run the test pass
`speakql-prototype.html`	To see the product before it exists
SpeakQL Project Report	Submission. §25 of the Backend Plan lists every amendment it still needs
SpeakQL Synopsis deck	Before the review presentation